

Codeforces Round 1114 (Div. 3)

A. Riptide

1 second, 256 megabytes

Alice, Bob, and Charlie are playing a game with tokens. They start with a , b , and c tokens, respectively.

The game is played in rounds. Before the beginning of each round, they check the number of tokens everyone has:

- If any two players have the exact same number of tokens, the game immediately ends.
- Otherwise, the round begins, all three players have a strictly different number of tokens. The player with the strictly most tokens gives exactly 1 token to the player with the strictly fewest tokens.

Given the starting tokens a , b , and c , determine exactly how many rounds the game will last before it ends.

Input

The first line contains a single integer t ($1 \leq t \leq 10^3$) — the number of test cases.

Each test case consists of a single line containing three integers a , b , and c ($1 \leq a, b, c \leq 10$).

Output

For each test case, output a single integer — the number of rounds the game will last before it ends.

input
6 1 2 3 4 6 1 3 3 7 1 7 10 6 1 9 1 1 1
output
1 2 0 3 3 0

In the first test case:

- No two players have the same number of tokens.
- Charlie has the most tokens (3 tokens), and Alice has the fewest tokens (1 token). Therefore, Charlie gives Alice a token.
- Now, Alice has 2 tokens, Bob has 2 tokens, and Charlie has 2 tokens. Since there are two players (or more) with the same number of tokens, the game ends.

The game ended after 1 round, so the answer is 1.

In the second test case, the game is played as follows:

- Bob gives Charlie a token, now Alice has 4 tokens, Bob has 5 tokens, and Charlie has 2 tokens.
- Bob gives Charlie a token, now Alice has 4 tokens, Bob has 4 tokens, and Charlie has 3 tokens. Since two players have the same number of tokens, the game ends.

The game lasted 2 rounds.

In the third test case, two players already have the same number of tokens. So the answer is 0 since no rounds were played.

B. Evanescent

2 seconds, 256 megabytes

Let $f(s)$ be the compressed version of a string s , formed by replacing every maximal contiguous block of identical characters with a single copy of that character. For example, $f(\text{"aabbcc"}) = \text{"abc"}$.

Let $|s|$ denote the length of a string s . Following this, $|f(s)|$ denotes the length of the compressed string. For example:

- $|f(\text{"aabbcc"})| = |\text{"abc"}| = 3$
- If the string is empty, its length is 0.

Yousef has given you a string s consisting of n lowercase Latin letters. You must delete **exactly** one character s_i ($2 \leq i \leq n - 1$) to form a new string s' , and then find the minimum possible value of $|f(s')|$.

Note that you **cannot** delete s_1 or s_n .

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($3 \leq n \leq 2 \cdot 10^5$) — the length of the string.

The second line of each test case contains a string s ($|s| = n$), consisting of lowercase Latin letters.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output a single integer — the minimum possible length of the resulting compressed string after deleting one character.

input
9 3 abb 3 aab 3 abc 4 abaa 4 abba 5 eeeeee 6 yysssee 7 abacaba 18 goodluckandhavefun
output
2 2 2 1 3 1 3 5 16

In the first test case, we can only delete the character $s_2 = \text{'b'}$, producing a string $s' = \text{"ab"}$, with $|f(s')| = 2$. Therefore, 2 is the minimum length achievable.

In the fourth test case, we can delete the character $s_2 = \text{'b'}$. The resulting string is $s' = \text{"aaa"}$ with $f(s') = \text{"a"}$, so $|f(s')| = 1$.

In the sixth test case, deleting any valid character results in $f(s') = "e"$ and $|f(s')| = 1$.

In the eighth test case, we can delete the character $s_4 = 'c'$. The resulting string is $s' = "abaaba"$ with $f(s') = "ababa"$ and $|f(s')| = 5$.

C1. Marenol (easy version)

2 seconds, 256 megabytes

This is the easy version of the problem. In this version, you are only asked to determine whether string a can be transformed into string b .

Yousef has given you two binary strings, a and b , of the same length n .

You are allowed to perform any of the following operations:

- Choose a substring* in a equal to **001** and replace it with **100**, or vice versa (i.e., **001** $\rightarrow$ **100** or **100** $\rightarrow$ **001**).
- Choose a substring in a equal to **110** and replace it with **011**, or vice versa (i.e., **011** $\rightarrow$ **110** or **110** $\rightarrow$ **011**).

Your task is to determine whether it is possible to transform string a into string b using a finite number of operations.

*A string a is a substring of a string b if a can be obtained from b by deletion of several (possibly zero or all) characters from the beginning and several (possibly zero or all) characters from the end.

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains a single integer n ($1 \leq n \leq 2 \cdot 10^5$) — the length of each string.

The second line of each test case contains a binary string a ($|a| = n$), consisting of only characters **0** and/or **1**.

The third line of each test case contains a binary string b ($|b| = n$), consisting of only characters **0** and/or **1**.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output "YES" if the string a can be transformed into string b using a finite number of operations, and "NO" otherwise.

You can output the answer in any case (upper or lower). For example, the strings "yEs", "yes", "Yes", and "YES" will be recognized as positive responses.

input
9
1
0
0
2
01
10
3
001
100
4
1010
0101
4
1100
1000
5
01001
10010
6
110000
000011
6
111000
000111
7
1001100
0000111
output
YES
NO
YES
NO
NO
YES
YES
NO
YES

In the first test case, it already holds that $a = b$. Therefore, the answer is YES.

In the second test case, we cannot perform any operation. Since $a \neq b$, the answer is NO.

In the third test case, we can choose the substring $a[1, 3] = 001$ and replace it with **100**, making $a = b$. Therefore, the answer is YES.

In the seventh test case, we can do the following in order:

- 110000** $\rightarrow$ **100100**
- 100100** $\rightarrow$ **100001**
- 100001** $\rightarrow$ **001001**
- 001001** $\rightarrow$ **000011**

Therefore, the answer is YES.

C2. Marenol (hard version)

2 seconds, 256 megabytes

This is the hard version of the problem. In this version, you are asked to determine the minimum number of operations to transform a into b .

Yousef has given you two binary strings, a and b , of the same length n .

You are allowed to perform any of the following operations:

- Choose a substring* in a equal to **001** and replace it with **100**, or vice versa (i.e. **001** $\rightarrow$ **100** or **100** $\rightarrow$ **001**).
- Choose a substring in a equal to **110** and replace it with **011**, or vice versa (i.e. **011** $\rightarrow$ **110** or **110** $\rightarrow$ **011**).

Your task is to determine the minimum number of operations required to transform string a into string b . If it is impossible to transform a into b using the given operations, output -1 instead.

*A string a is a substring of a string b if a can be obtained from b by deletion of several (possibly zero or all) characters from the beginning and several (possibly zero or all) characters from the end.

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains a single integer n ($1 \leq n \leq 2 \cdot 10^5$) — the length of each string.

The second line of each test case contains a binary string a ($|a| = n$), consisting of only characters 0 and/or 1.

The third line of each test case contains a binary string b ($|b| = n$), consisting of only characters 0 and/or 1.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output the minimum number of operations required to transform a into b . If it is impossible, output -1 instead.

input
5 4 0100 0001 4 0100 0010 6 110000 000011 8 10101010 10101010 5 01001 10010
output
1 -1 4 0 3

In the first test case, we can choose the substring $a[2, 4] = 100$ and replace it with 001 . This takes exactly 1 operation.

In the second test case, it is impossible to transform a into b , so the answer is -1 .

In the third test case, we can do the following in order:

- 110000 $\rightarrow$ 100100
- 100100 $\rightarrow$ 100001
- 100001 $\rightarrow$ 001001
- 001001 $\rightarrow$ 000011

This takes 4 operations. It can be shown that 4 is the minimum answer.

D. Silhouette

2.5 seconds, 256 megabytes

Yousef has a secret array a of n strictly positive integers. For each element a_i , its *shadow* b_i is the sum of all elements in a that are strictly smaller than a_i . Formally:

$$b_i = \sum_{\substack{1 \leq j \leq n \\ a_j < a_i}} a_j$$

You are given the shadow array b . Your task is to reconstruct the **lexicographically smallest** valid array a consisting of strictly positive integers that satisfies the above condition. If no such array exists, output -1 .

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($1 \leq n \leq 2 \cdot 10^5$) — the size of the array.

The second line of each test case contains n integers $b_1, b_2, \dots, b_n$ ($0 \leq b_i \leq 2 \cdot 10^{14}$) — the shadow array.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^{18}$) — the lexicographically smallest valid array a that satisfies the condition. If no valid array exists, output -1 instead.

input
8 1 0 5 0 4 0 4 14 3 4 0 0 3 0 0 0 3 0 1 1 4 1 1 1 1 7 0 4 4 4 4 4 9 5 0 0 0 3 3
output
1 2 5 2 5 6 3 2 2 1 1 1 1 2 2 -1 -1 1 1 1 2 2

In the first test case, the answer is $a = [1]$. Since there is only one element, there are no strictly smaller elements, so its shadow is 0. Thus $a = [1]$ is valid. It is also lexicographically smallest, because the only allowed values are positive integers, and 1 is the smallest possible.

In the second test case, the answer is $a = [2, 5, 2, 5, 6]$:

- For each 2, there is no smaller element in the array, so the shadow is 0.
- For each 5, the strictly smaller elements are the two 2's, so the shadow is $2 + 2 = 4$.
- For 6, the strictly smaller elements are two 2's and two 5's, so the shadow is $2 + 2 + 5 + 5 = 14$.

Therefore the shadow array is exactly $b = [0, 4, 0, 4, 14]$.

In the third test case, the shadow array for $a = [3, 2, 2]$ is calculated as follows:

- For $a_1 = 3$, the strictly smaller elements in the array are the two 2s. Their sum is $2 + 2 = 4$. So, $b_1 = 4$.
- For $a_2 = 2$, there are no strictly smaller elements in the array. So, $b_2 = 0$.
- For $a_3 = 2$, there are no strictly smaller elements in the array. So, $b_3 = 0$.

The resulting shadow array is $b = [4, 0, 0]$, which matches the input.

2 seconds, 256 megabytes

E. Chronostasis

2 seconds, 256 megabytes

Yousef has a hidden array a of length n consisting entirely of **strictly positive** integers.

An operation was performed exactly once to create an array b :

- Set $b_1 = a_1$.
- For every i from 2 to n , set $b_i = a_i - a_{i-1}$.
- After this, the elements of b were completely shuffled.

You are given the shuffled array b . Reconstruct the **lexicographically smallest** original array a . If it's impossible for any arrangement of b to produce an array a of strictly positive integers, output -1 .

Input

The first line of input contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($1 \leq n \leq 2 \cdot 10^5$) — the size of the array.

The second line of each test case contains n integers $b_1, b_2, \dots, b_n$ ($-10^9 \leq b_i \leq 10^9$) — the elements of the shuffled array b .

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output n strictly positive integers $a_1, a_2, \dots, a_n$ ($a_i \geq 1$) — the lexicographically smallest original array a . If it's impossible to create a valid array a , output -1 instead.

input
8 1 5 4 -5 2 1 1 6 -3 4 2 -1 1 0 6 -2 -2 4 1 0 1 7 0 0 -2 3 0 -1 2 8 -1 -1 -1 -1 5 0 0 1 5 1000000000 500000000 750000000 100000000 900000000 10 1000000000 -1000000000 500000000 -500000000 1 1 -1 -1 2 -2
output
5 -1 1 1 3 2 6 3 1 1 2 6 4 2 2 1 1 1 1 4 2 1 1 1 6 5 4 3 2 100000000 600000000 135000000 225000000 325000000 -1

In the first test case, the only valid array is $a = [5]$.

In the second test case, there is no valid arrangement of the elements of b that reconstructs an array a consisting entirely of strictly positive integers. Therefore, the answer is -1 .

In the third test case, one valid arrangement reconstructs the array $a = [1, 1, 3, 2, 6, 3]$. The resulting sequence of differences $[1, 0, 2, -1, 4, -3]$ is a permutation of the given array b , and among all valid reconstructions, this array is lexicographically smallest.

Yousef has given you an **even** integer n and two arrays, a and b , both consisting of n non-negative integers.

You can perform the following operation on array a any number of times (possibly zero):

- Choose an index i ($1 \leq i \leq n$).
- For all j such that $1 \leq j \leq n$ and $j \neq i$, replace a_j with $a_j \oplus a_i$. (Here, $\oplus$ denotes the **bitwise XOR operation**).
- The element a_i remains unchanged.

Determine whether it is possible to transform array a into array b using a finite number of operations.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases. The description of the test cases follows.

The first line of each test case contains an **even** integer n ($2 \leq n \leq 2 \cdot 10^5$) — the length of the array.

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($0 \leq a_i < 2^{30}$) — the elements of the array a .

The third line of each test case contains n integers $b_1, b_2, \dots, b_n$ ($0 \leq b_i < 2^{30}$) — the elements of the array b .

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output "YES" if the array a can be transformed into array b using a finite number of operations, and "NO" otherwise.

You can output the answer in any case (upper or lower). For example, the strings "yEs", "yes", "Yes", and "YES" will be recognized as positive responses.

input
6 2 1 2 1 0 4 1 2 4 7 6 7 5 3 4 1 2 4 8 8 4 2 1 4 1 2 3 4 1 2 4 5 4 1 2 0 3 3 3 0 3 6 3 5 6 9 10 12 6 5 3 12 15 9
output
NO YES YES NO NO YES

In the first test case, starting from $[1, 2]$, there is no sequence of operations that ends at $[1, 0]$, so the answer is "NO".

In the second test case, one valid sequence is:

- Choose index 2, $[1, 2, 4, 7] \rightarrow [3, 2, 6, 5]$.
- Choose index 3, $[3, 2, 6, 5] \rightarrow [5, 4, 6, 3]$.
- Choose index 4, $[5, 4, 6, 3] \rightarrow [6, 7, 5, 3]$.

In the third test case, one valid sequence is:

- Choose index 1, [1, 2, 4, 8] → [1, 3, 5, 9].
- Choose index 2, [1, 3, 5, 9] → [2, 3, 6, 10].
- Choose index 3, [2, 3, 6, 10] → [4, 5, 6, 12].
- Choose index 2, [4, 5, 6, 12] → [1, 5, 3, 9].
- Choose index 4, [1, 5, 3, 9] → [8, 12, 10, 9].
- Choose index 1, [8, 12, 10, 9] → [8, 4, 2, 1].

G. Nightcrawler

2.5 seconds, 256 megabytes

Yousef has given you a rooted tree* with n vertices, where the root is vertex 1. Each vertex i is assigned an integer a_i .

You must partition the set of all n vertices into exactly k disjoint subsets $S_1, S_2, \dots, S_k$ (that is, each vertex must belong to exactly one of the k sets) such that the following condition is satisfied:

- For any subset S_i containing two or more vertices, for every pair of vertices $u, v \in S_i$, one must be the ancestor† of the other (i.e., they must all lie on the same path extending from the root toward a leaf).

The score of a subset S_i is defined as the maximum value a_u among all vertices u in that subset. The score of the partition is the sum of the scores of the k subsets. In other words, the score of the partition is equal

to $\sum_{i=1}^k \max_{u \in S_i} a_u$.

For every integer k from 1 to n , calculate the maximum possible score of a partition. If it is impossible to partition the tree into exactly k subsets that satisfy the condition, output -1 .

*A tree is a connected graph without cycles. A rooted tree is a tree where one vertex is special and called the root.

†An ancestor of vertex v is any vertex on the simple path from v to the root, including the root, but not including v . The root has no ancestors.

Input

The first line contains an integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains an integer n ($3 \leq n \leq 2 \cdot 10^5$) — the number of vertices.

The second line of each test case contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) — the values of the vertices.

The third line of each test case contains $n - 1$ integers $p_2, p_3, \dots, p_n$ ($1 \leq p_i < i$), where p_i is the parent of the i -th vertex.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

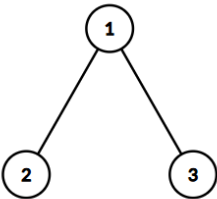
Output

For each test case, output a single line containing n space-separated integers. The k -th integer should represent the maximum possible score for a partition of k subsets. If it is impossible to partition the tree into k subsets, output -1 for that value.

input
7 3 10 20 30 1 1 4 5 10 15 20 1 2 2 4 1 2 3 4 1 1 3 5 1 2 3 4 5 1 2 3 4 9 1 100 1 90 80 1 2 3 4 1 1 2 4 3 3 3 3 6 5 4 10 3 9 1 1 2 1 4 1 4 10 10 20 1 1 2 1
output
-1 50 60 -1 35 45 50 -1 6 9 10 5 9 12 14 15 -1 -1 -1 -1 110 200 280 281 282 -1 -1 24 28 31 32 -1 30 40 41

In the first test case:

- For $k = 1$, we would need all the vertices to be in the same set. However, for vertices 2 and 3, neither of them is the ancestor of the other. Therefore, there is no valid partition.
- For $k = 2$, we can make $S_1 = \{2\}$, $S_2 = \{1, 3\}$. The score of this partition is $\max_{u \in S_1} a_u + \max_{u \in S_2} a_u = 20 + 30 = 50$. It can be shown that this is the maximum score.
- For $k = 3$, we can make $S_1 = \{1\}$, $S_2 = \{2\}$, $S_3 = \{3\}$. The score of this partition is $10 + 20 + 30 = 60$.

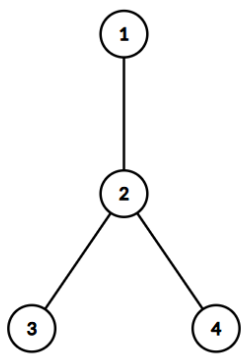


The given tree in the first test case.

In the second test case:

- For $k = 1$, we would need all the vertices to be in the same set. However, vertices 3 and 4 are not on the same root-to-leaf path, so this is impossible.
- For $k = 2$, we can make $S_1 = \{3\}$, $S_2 = \{1, 2, 4\}$. The score of this partition is $\max_{u \in S_1} a_u + \max_{u \in S_2} a_u = 15 + 20 = 35$. It can be shown that this is the maximum score.
- For $k = 3$, we can make $S_1 = \{3\}$, $S_2 = \{4\}$, $S_3 = \{1, 2\}$. The score of this partition is $15 + 20 + 10 = 45$. It can be shown that this is the maximum score.
- For $k = 4$, we can make $S_1 = \{1\}$, $S_2 = \{2\}$, $S_3 = \{3\}$, $S_4 = \{4\}$. The score of this partition is $5 + 10 + 15 + 20 = 50$.

The given tree in the second test case.



Hints: Codeforces Round 1114 (Div. 3)

Taken from the official editorial, not written by us. Read one hint at a time.

A. Riptide (official hint)

Hint: Assume $a \leq b \leq c$, does b ever change?

B. Evanescent (official hint)

Hint: If we perform the operation on any character in a maximal contiguous block of length > 1 , then $f(s') = f(s)$.

C1. Marenol (easy version) (official hints)

Hint 1: The operation is equivalent to freely swapping a_i with a_{i+2} for any valid index i .

Hint 2: After the reduction from Hint 1, we can treat a as two separate strings.

C2. Marenol (hard version) (editorial, first line)

Building off the core observation from the easy version, we know that elements are strictly trapped in their own parity.

D. Silhouette (official hints)

Hint 1: Equal values in a will always have the exact same shadow, and larger elements always have strictly larger shadows. So if you sort the distinct values of b , they correspond in order to the distinct values of a : the group with the i -th smallest value has shadow equal to the i -th smallest distinct value in b .

Hint 2: Let $s_1 < s_2 < \dots < s_m$ be the distinct sorted values of b , and let group i be all elements sharing the value corresponding to s_i . Then s_i is the sum of all groups smaller than group i , and s_{i+1} is that same sum plus group i itself. So $s_{i+1} - s_i = c_i \cdot v_i$, where c_i is the count of elements in group i and v_i is their value.

Hint 3: What must s_1 be? And once every v_i is pinned down by the equation above, is there a group left with no equation constraining its value? What's the smallest legal choice for it?

E. Chronostasis (official hints)

Hint 1: The problem states that $b_i = a_i - a_{i-1}$. This directly implies that $a_i = a_{i-1} + b_i$. If you assume $a_0 = 0$, you can view a simply as the prefix sums of some permutation of b .

Hint 2: The standard way to achieve a lexicographically smallest sequence is to build it greedily from left to right, making a_1 as small as possible, then a_2 as small as possible, and so forth.

Hint 3: Since a_{i-1} is fixed from the previous steps, minimizing a_i means you must pick the smallest available b_i . However, the problem imposes a strict rule: every element in a must be strictly positive ($a_i \geq 1$). How does this restrict your choice of b_i ?

F. Whiplash (official hints)

Hint 1: Try tracing the array elements after performing two consecutive operations, say on index i and then on index j . If you look at the set of resulting values and ignore their order, how does it compare to the set of values you would get if you only performed a single operation on index j ?

Hint 2: Any sequence of operations results in an array that is just a permutation of either the original array a , or the array a after exactly one operation.

G. Nightcrawler (official hints)

Hint 1: Because no subset can contain branching paths, two distinct leaves can never belong to the same subset. If the tree has ℓ leaves, you need at least ℓ subsets. Any $k < \ell$ is impossible. For $k \geq \ell$, the condition is always satisfiable since you can always break paths into smaller singletons.

Hint 2: Try to build the solution bottom-up. Each leaf starts a path. When these paths meet at a parent node, the parent can only join one of them. To maximize the total score, which path's maximum should the parent node interact with?

Hint 3: It is always optimal for the parent to join the path that currently has the smallest maximum. If the parent's value is larger, it successfully increases that path's maximum, leaving the old maximum behind as a spare value. If the parent is smaller, the path is unaffected, and the parent itself becomes the spare value that we can use if we are allowed extra subsets.

Source: <https://codeforces.com/blog/entry/155666>